

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Лекция - 8***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Лекция – 8
Основные подходы к интегрированию программных модулей

ОСНОВЫ ПРОЕКТИРОВАНИЯ ПРОГРАММНЫХ СИСТЕМ

В этой главе рассматривается содержание этапа проектирования и его место в жизненном цикле конструирования программных систем. Дается обзор архитектурных моделей ПО, обсуждаются классические проектные характеристики: модульность, информационная закрытость, сложность, связность, сцепление и метрики для их оценки.

Особенности процесса синтеза программных систем

Известно, что технологический цикл конструирования программной системы (ПС) включает три процесса — анализ, синтез и сопровождение.

В ходе анализа ищется ответ на вопрос: «Что должна делать будущая система?». Именно на этой стадии закладывается фундамент успеха всего проекта. Известно множество неудачных реализаций из-за неполноты и неточностей в определении требований к системе.

В процессе синтеза формируется ответ на вопрос: «Каким образом система будет реализовывать предъявляемые к ней требования?». Выделяют три этапа синтеза: проектирование ПС, кодирование ПС, тестирование ПС (рис. 4.1).

Рассмотрим информационные потоки процесса синтеза.

Этап проектирования питают требования к ПС, представленные информационной, функциональной и поведенческой моделями анализа. Иными словами, модели анализа поставляют этапу проектирования исходные сведения для работы. Информационная модель описывает информацию, которую, по мнению заказчика, должна обрабатывать ПС. Функциональная модель определяет перечень функций обработки. Поведенческая модель фиксирует желаемую динамику системы (режимы ее работы). На выходе этапа проектирования — разработка данных, разработка архитектуры и процедурная разработка ПС.

Разработка данных — это результат преобразования информационной модели анализа в структуры данных, которые потребуются для реализации программной системы.

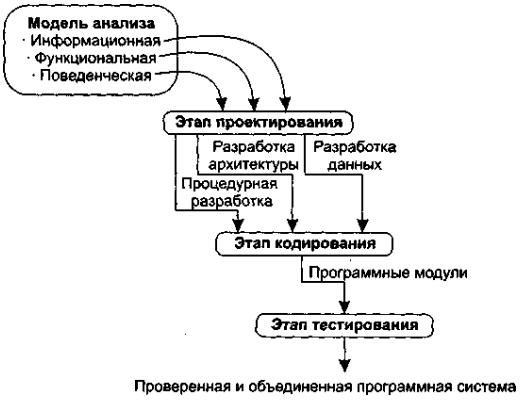


Рис. 4.1. Информационные потоки процесса синтеза ПС

Разработка архитектуры выделяет основные структурные компоненты и фиксирует связи между ними.

Процедурная разработка описывает последовательность действий в структурных компонентах, то есть определяет их содержание.

Далее создаются тексты программных модулей, проводится тестирование для объединения и проверки ПС. На проектирование, кодирование и тестирование приходится более 75% стоимости конструирования ПС. Принятые здесь решения оказывают решающее воздействие на успех реализации ПС и легкость, с которой ПС будет сопровождаться.

Следует отметить, что решения, принимаемые в ходе проектирования, делают его стержневым этапом процесса синтеза. Важность проектирования можно определить одним словом — качество. Проектирование — этап, на котором

«выращивается» качество разработки ПС. Справедлива следующая аксиома разработки: может быть плохая ПС при хорошем проектировании, но не может быть хорошей ПС при плохом проектировании. Проектирование обеспечивает нас такими представлениями ПС, качество которых можно оценить. Проектирование — единственный путь, обеспечивающий правильную трансляцию требований заказчика в конечный программный продукт.

Особенности этапа проектирования

Проектирование — итерационный процесс, при помощи которого требования к ПС транслируются в инженерные представления ПС. Вначале эти представления дают только концептуальную информацию (на высоком уровне абстракции), последующие уточнения приводят к формам, которые близки к текстам на языках программирования.

Обычно в проектировании выделяют две ступени: предварительное проектирование и детальное проектирование. Предварительное проектирование формирует абстракции архитектурного уровня, детальное проектирование уточняет эти абстракции, добавляет подробности алгоритмического уровня. Кроме того, во многих случаях выделяют интерфейсное проектирование, цель которого — сформировать графический интерфейс пользователя (GUI). Схема информационных связей процесса проектирования приведена на рис. 4.2.

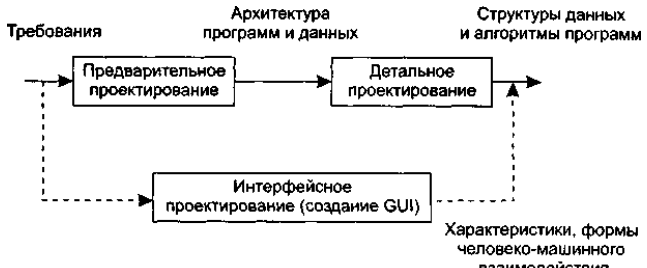


Рис. 4.2. Информационные связи процесса проектирования Предварительное проектирование обеспечивает:

идентификацию подсистем;

определение основных принципов управления подсистемами, взаимодействия подсистем. Предварительное проектирование включает три типа деятельности:

Структурирование системы. Система структурируется на несколько подсистем, где под подсистемой понимается независимый программный компонент. Определяются взаимодействия подсистем.

Моделирование управления. Определяется модель связей управления между частями системы.

Декомпозиция подсистем на модули. Каждая подсистема разбивается на модули. Определяются типы модулей и межмодульные соединения.

Рассмотрим вопросы структурирования, моделирования и декомпозиции более подробно.

Структурирование системы

Известны четыре модели системного структурирования:

- модель хранилища данных;
- модель клиент-сервер;
- трехуровневая модель;
- модель абстрактной машины.

В модели хранилища данных (рис. 4.3) подсистемы разделяют данные, находящиеся в общей памяти. Как правило, данные образуют БД. Предусматривается система управления этой базой.

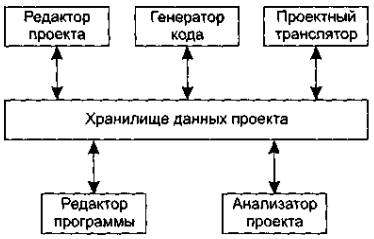


Рис. 4.3. Модель хранилища данных

Модель клиент-сервер используется для распределенных систем, где данные распределены по серверам (рис. 4.4). Для передачи данных применяют сетевой протокол, например TCP/IP.

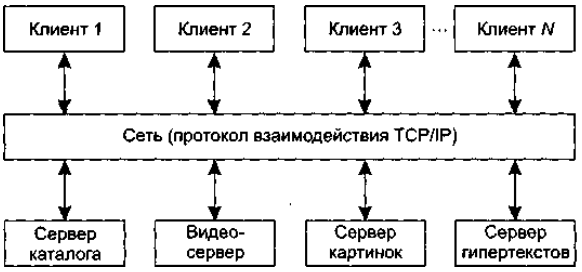


Рис. 4.4. Модель клиент-сервер

Трехуровневая модель является развитием модели клиент-сервер (рис. 4.5).



Рис. 4.5. Трехуровневая модель

Уровень графического интерфейса пользователя запускается на машине клиента. Бизнес-логику образуют модули, осуществляющие функциональные обязанности системы. Этот уровень запускается на сервере приложения. Реляционная СУБД хранит

данные, требуемые уровню бизнес-логики. Этот уровень запускается на втором сервере — сервере базы данных.

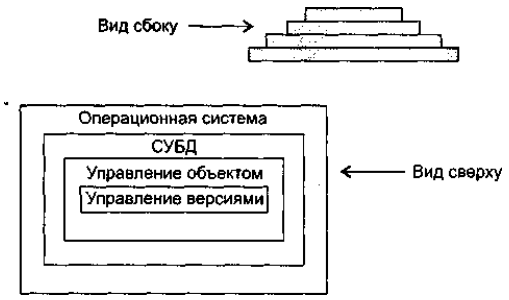
Преимущества трехуровневой модели:

упрощается такая модификация уровня, которая не влияет на другие уровни;

отделение прикладных функций от функций управления БД упрощает оптимизацию всей системы.

Модель абстрактной машины отображает многослойную систему (рис. 4.6).

Каждый текущий слой реализуется с использованием средств, обеспечиваемых



слоем-фундаментом.

Рис. 4.6. Модель абстрактной машины

Моделирование управления

Известны два типа моделей управления:

модель централизованного управления;

модель событийного управления.

В модели централизованного управления одна подсистема выделяется как системный контроллер. Ее обязанности — руководить работой других подсистем. Различают две разновидности моделей централизованного управления: модель вызов-возврат (рис. 4.7) и Модель менеджера (рис. 4.8), которая используется в системах параллельной обработки.

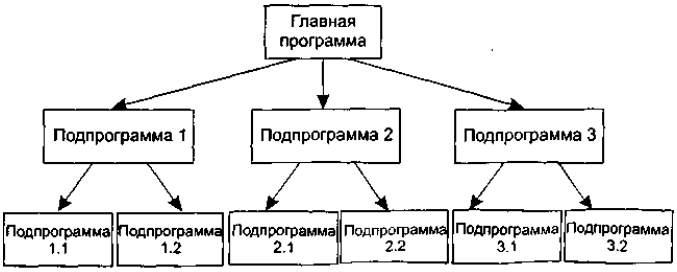
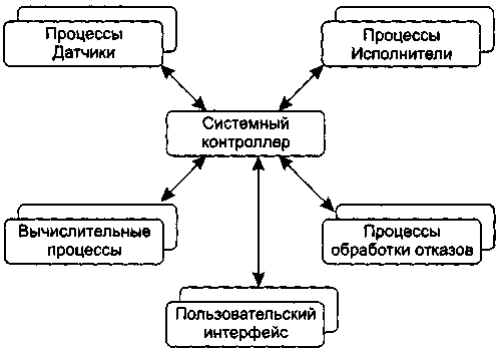


Рис. 4.7. Модель вызов-возврат

В модели событийного управления системой управляют внешние события. Используются две разновидности модели

событийного управления: широковещательная модель и модель, управляемая



прерываниями.

Рис. 4.8. Модель менеджера

В широковещательной модели (рис. 4.9) каждая подсистема уведомляет обработчика о своем интересе к конкретным событиям. Когда событие происходит, обработчик пересылает его подсистеме, которая может обработать это событие. Функции управления в обработчик не встраиваются.



Рис. 4.9. Широковещательная модель

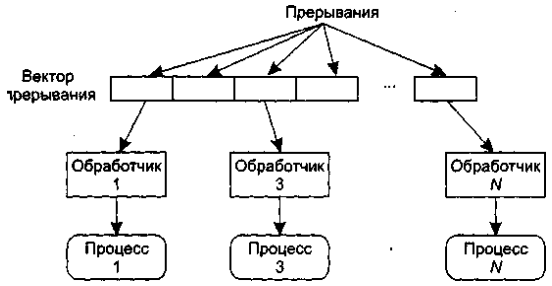


Рис. 4.10. Модель, управляемая прерываниями

В модели, управляемой прерываниями (рис. 4.10), все прерывания разбиты на группы — типы, которые образуют вектор прерываний. Для каждого типа прерывания есть свой обработчик. Каждый обработчик реагирует на свой тип прерывания и запускает свой процесс.

Декомпозиция подсистем на модули

Известны два типа моделей модульной декомпозиции:
модель потока данных;
модель объектов.

В основе модели потока данных лежит разбиение по функциям.

Модель объектов основана на слабо сцепленных сущностях, имеющих собственные наборы данных, состояния и наборы операций.

Очевидно, что выбор типа декомпозиции должен определяться сложностью разбиваемой подсистемы.